

Pokedex project for week6

Step 1: project setup

We're starting the project fresh today. Everything below assumes Node.js is already installed.

Scaffold the project

```
npm create vite@latest pokedex-mini -- --template react
cd pokedex-mini
npm install
```

Open the folder in VS Code. Vite's default template comes with a demo counter app and some starter assets we don't need — you can delete `src/assets/` and `src/App.css` now, we'll write our own minimal styles.

Where today's files live

```
pokedex-mini/
├── index.html
├── package.json
├── vite.config.js
├── src/
│   ├── main.jsx          ← keep Vite's default, shown below for
reference
│   ├── App.jsx           ← replace the default content
│   ├── index.css         ← replace with our own minimal styles
│   └── components/
│       └── PokemonList.jsx ← new file, create this folder and file
```

`src/main.jsx` (Vite's default — leave as is)

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

`src/App.jsx` (replace the default content with this)

```
import PokemonList from "../components/PokemonList.jsx";

function App() {
  return (
    <div className="app">
      <h1>PokéDex Mini</h1>
      <PokemonList />
    </div>
  );
}

export default App;
```

src/index.css (replace the default content with this)

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: system-ui, -apple-system, "Segoe UI", sans-serif;
  background: #f5f5f5;
  color: #222;
}

.app {
  max-width: 480px;
  margin: 0 auto;
  padding: 32px 16px;
}

h1 {
  text-align: center;
  color: #cc0000;
}

.status {
  text-align: center;
  font-size: 18px;
  color: #555;
}

.status-error {
  color: #b00020;
}

.pokemon-list {
  list-style: none;
  padding: 0;
```

```
margin: 24px 0 0;
display: flex;
flex-direction: column;
gap: 8px;
}

.pokemon-list-item {
display: flex;
align-items: center;
gap: 12px;
background: white;
border-radius: 8px;
padding: 12px 16px;
box-shadow: 0 1px 2px rgba(0, 0, 0, 0.08);
}

.pokemon-sprite {
flex-shrink: 0;
background: #f0f0f0;
border-radius: 6px;
}

.pokemon-id {
font-family: monospace;
color: #888;
min-width: 48px;
}

.pokemon-name {
font-weight: 600;
text-transform: capitalize;
}
```

`src/components/PokemonList.jsx` (new file — create the `components` folder first)

This is the full component, combining everything from Sections 3–4, plus three small helpers: one to pull the Pokédex number out of the API's `url` field (so we can show "#001 Bulbasaur" instead of just the name), one to capitalize the name, and one to build an image URL from that same number.

Why there's no extra fetch for the images: PokéAPI's sprite artwork is hosted as plain image files on GitHub, at a predictable address — `.../sprites/pokemon/{id}.png`. Since we already pull `{id}` out of `pokemon.url` for the badge, we get the picture for free: no second request, no extra loading state, just a URL built from data we already have in state.

```
import { useState, useEffect } from "react";

function getIdFromUrl(url) {
  // url looks like "https://pokeapi.co/api/v2/pokemon/25/"
  const parts = url.split("/").filter(Boolean); // drop empty strings from
  the trailing slash
  return parts[parts.length - 1];
}
```

```
}

function capitalize(name) {
  return name.charAt(0).toUpperCase() + name.slice(1);
}

function getSpriteUrl(id) {
  return
  `https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/
  ${id}.png`;
}

function PokemonList() {
  const [pokemons, setPokemons] = useState([]);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    async function loadPokemons() {
      setIsLoading(true);
      setError(null);

      try {
        const response = await fetch("https://pokeapi.co/api/v2/pokemon?
        limit=20");

        if (!response.ok) {
          throw new Error(`Server responded with status
          ${response.status}`);
        }

        const data = await response.json();
        setPokemons(data.results); // [{ name, url }, ...]
      } catch (err) {
        setError(err.message);
      } finally {
        setIsLoading(false);
      }
    }

    loadPokemons();
  }, []); // empty array: fetch once, when the component mounts

  if (isLoading) {
    return <p className="status">Loading Pokémon...</p>;
  }

  if (error) {
    return <p className="status status-error">Couldn't load the list:
    {error}</p>;
  }

  return (
    <ul className="pokemon-list">
```

```

    {pokemons.map((pokemon) => {
      const id = getIdFromUrl(pokemon.url);
      return (
        <li key={pokemon.name} className="pokemon-list-item">
          <img
            className="pokemon-sprite"
            src={getSpriteUrl(id)}
            alt={pokemon.name}
            width={48}
            height={48}
          />
          <span className="pokemon-id">#{id.padStart(3, "0")}</span>
          <span className="pokemon-name">{capitalize(pokemon.name)}
        </span>
        </li>
      );
    })}
  </ul>
);
}

export default PokemonList;

```

A couple of details worth pointing out here:

- `id` is computed once per Pokémon and reused for both the image and the badge, instead of calling `getIdFromUrl(pokemon.url)` twice — a small habit worth building early.
- If you want to guard against the rare missing sprite file, you can add `onError={(e) => { e.target.style.visibility = "hidden"; }}` to the `<img>` — optional, and a nice "what does this even do" discussion prompt if a curious student asks, but not required for the milestone.

Run it

```
npm run dev
```

Open the printed local address. You should briefly see "**Loading Pokémon...**", then a list of rows, each with a small sprite image next to its number and name, like:

```

[img] #001  Bulbasaur
[img] #002  Ivysaur
[img] #003  Venusaur
[img] #004  Charmander
...
[img] #020  Raticate

```

Try this yourself: temporarily change the URL to something broken (e.g.

<https://pokeapi.co/api/v3/pokemon>) and confirm the error message shows up. Then fix it back and

confirm the list returns. If you can make both states appear on demand, you understand the pattern — and you're ready for the milestone check.

Step 2: adding the search feature

We're extending last session's project — no need to start over.

What's new today

```
pokedex-mini/  
└─ src/  
    ├── App.jsx           ← updated: renders SearchForm too  
    ├── index.css         ← updated: styles for the search UI  
    └─ components/  
        ├── PokemonList.jsx ← unchanged from S21  
        └── SearchForm.jsx  ← new file
```

`src/components/SearchForm.jsx` (new file)

```
import { useState } from "react";  
  
function capitalize(name) {  
  return name.charAt(0).toUpperCase() + name.slice(1);  
}  
  
function SearchForm() {  
  const [query, setQuery] = useState("");  
  const [result, setResult] = useState(null);  
  const [isLoading, setIsLoading] = useState(false);  
  const [error, setError] = useState(null);  
  
  async function handleSubmit(event) {  
    event.preventDefault(); // stop the browser's default full-page reload  
  
    const name = query.trim().toLowerCase();  
  
    if (name === "") {  
      setError("Type a Pokémon name first.");  
      setResult(null);  
      return;  
    }  
  
    setIsLoading(true);  
    setError(null);  
    setResult(null); // clear the old result so a slow new search doesn't  
    show stale data mid-flight  
  
    try {  
      const response = await
```

```

fetch(`https://pokeapi.co/api/v2/pokemon/${name}`);

    if (!response.ok) {
        throw new Error(`No Pokémon named "${name}" – check the
spelling.`);
    }

    const data = await response.json();
    setResult(data);
} catch (err) {
    setError(err.message);
} finally {
    setIsLoading(false);
}
}

return (
    <div className="search">
        <form onSubmit={handleSubmit} className="search-form">
            <input
                type="text"
                value={query}
                onChange={(event) => setQuery(event.target.value)}
                placeholder="Search a Pokémon by name..."
                className="search-input"
            />
            <button type="submit" className="search-button">
                Search
            </button>
        </form>

        {isLoading && <p className="status">Looking up {query}...</p>}
        {error && <p className="status status-error">{error}</p>}

        {result && (
            <div className="search-result">
                <img
                    src={result.sprites.front_default}
                    alt={result.name}
                    width={64}
                    height={64}
                />
                <div>
                    <p className="pokemon-name">{capitalize(result.name)}</p>
                    <p className="pokemon-types">
                        {result.types.map((t) => t.type.name).join(", ")}
                    </p>
                </div>
            </div>
        )}
    </div>
);
}

```

```
export default SearchForm;
```

Notice what's *not* here: no `getIdFromUrl`, no `getSpriteUrl` helper. Last session's list endpoint only gave us a `name` and a `url`, so we had to build our own image address from the ID we extracted. This session's **detail** endpoint (`/pokemon/{name}`) hands back a full `sprites.front_default` URL directly, plus a ready-to-use `id`, `types` array, and much more. Different endpoints return different shapes — always check what's actually in the response before writing a workaround for something the API might already give you.

`src/App.jsx` (add `SearchForm`)

```
import SearchForm from "../components/SearchForm.jsx";
import PokemonList from "../components/PokemonList.jsx";

function App() {
  return (
    <div className="app">
      <h1>PokéDex Mini</h1>
      <SearchForm />
      <PokemonList />
    </div>
  );
}

export default App;
```

`src/index.css` (append these rules)

```
.search {
  margin-bottom: 24px;
}

.search-form {
  display: flex;
  gap: 8px;
}

.search-input {
  flex: 1;
  padding: 8px 12px;
  border: 1px solid #ccc;
  border-radius: 6px;
  font-size: 16px;
}

.search-button {
  padding: 8px 16px;
```



```
border: none;
border-radius: 6px;
background: #cc0000;
color: white;
font-weight: 600;
cursor: pointer;
}

.search-button:hover {
  background: #a00000;
}

.search-result {
  display: flex;
  align-items: center;
  gap: 12px;
  margin-top: 12px;
  background: white;
  border-radius: 8px;
  padding: 12px 16px;
  box-shadow: 0 1px 2px rgba(0, 0, 0, 0.08);
}

.pokemon-types {
  color: #666;
  text-transform: capitalize;
  margin: 4px 0 0;
}
```

Run it

```
npm run dev
```

Type **pikachu** and submit — you should see its sprite, "Pikachu", and "electric" appear above the list. Try these too:

- **Submit with the box empty** → "Type a Pokémon name first." (validation error, no fetch happens at all)
- **Type **pikachuu** and submit** → "No Pokémon named "pikachuu" — check the spelling." (a real fetch happened and got a 404)
- **Type **Charizard** (capitalized) and submit** → still works, because `.toLowerCase()` normalized it before the request

Try this yourself: temporarily remove the `if (name === '')` check and submit an empty form. Watch what happens in the Network tab — you'll send a request to `.../pokemon/` with nothing after it, and get a 404 you didn't need to cause. That's exactly what validation is for: catching the bad case *before* it becomes a wasted network request.

Step 3: adding routing

What's new today

```
pokedex-mini/
├── src/
│   ├── App.jsx           ← rewritten: now defines routes instead
│   └── index.css         ← updated: styles for links, detail
├── of rendering pages directly
├── page, 404
├── components/
│   ├── PokemonList.jsx   ← updated: list items become <Link>s
│   └── SearchForm.jsx    ← simplified: navigates instead of
├── fetching
├── pages/                ← new folder
│   ├── ListPage.jsx      ← new
│   ├── DetailPage.jsx    ← new
│   └── NotFoundPage.jsx   ← new
```

Install the router first:

```
npm install react-router-dom
```

src/App.jsx (rewrite)

```
import { HashRouter, Routes, Route } from "react-router-dom";
import ListPage from "./pages/ListPage.jsx";
import DetailPage from "./pages/DetailPage.jsx";
import NotFoundPage from "./pages/NotFoundPage.jsx";

function App() {
  return (
    <HashRouter>
      <div className="app">
        <h1>PokéDex Mini</h1>
        <Routes>
          <Route path="/" element={<ListPage />} />
          <Route path="/pokemon/:name" element={<DetailPage />} />
          <Route path="*" element={<NotFoundPage />} />
        </Routes>
      </div>
    </HashRouter>
  );
}

export default App;
```

src/pages/ListPage.jsx (new — this is what used to be directly inside App.jsx)

```
import SearchForm from "../components/SearchForm.jsx";
import PokemonList from "../components/PokemonList.jsx";

function ListPage() {
  return (
    <>
      <SearchForm />
      <PokemonList />
    </>
  );
}

export default ListPage;
```

src/pages/DetailPage.jsx (new)

```
import { useState, useEffect } from "react";
import { useParams, Link } from "react-router-dom";

function capitalize(name) {
  return name.charAt(0).toUpperCase() + name.slice(1);
}

function DetailPage() {
  const { name } = useParams();
  const [pokemon, setPokemon] = useState(null);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    let isCurrent = true;

    async function loadPokemon() {
      setIsLoading(true);
      setError(null);
      setPokemon(null);

      try {
        const response = await
fetch(`https://pokeapi.co/api/v2/pokemon/${name}`);

        if (!response.ok) {
          throw new Error(`No Pokémon named "${name}" – check the
spelling.`);
        }

        const data = await response.json();
```

```

        if (isCurrent) {
            setPokemon(data);
        }
    } catch (err) {
        if (isCurrent) {
            setError(err.message);
        }
    } finally {
        if (isCurrent) {
            setIsLoading(false);
        }
    }
}

loadPokemon();

return () => {
    isCurrent = false;
};
}, [name]); // re-run whenever the :name in the URL changes

if (isLoading) return <p className="status">Loading {name}...</p>;
if (error) return <p className="status status-error">{error}</p>;

return (
    <div className="detail-page">
        <Link to="/" className="back-link">← Back to list</Link>
        <img
            src={pokemon.sprites.other["official-artwork"].front_default}
            alt={pokemon.name}
            width={200}
            height={200}
        />
        <h2>{capitalize(pokemon.name)}</h2>
        <p className="pokemon-types">
            {pokemon.types.map((t) => t.type.name).join(", ")}
        </p>
        <ul className="stat-list">
            {pokemon.stats.map((s) => (
                <li key={s.stat.name}>
                    <span className="stat-name">{s.stat.name}</span>
                    <span className="stat-value">{s.base_stat}</span>
                </li>
            ))}
        </ul>
    </div>
);
}

export default DetailPage;

```

Note the bigger, nicer `sprites.other["official-artwork"].front_default` image here — PokéAPI's detail response actually includes several sprite variants; this is a good moment to have students open the raw JSON (visit the API URL directly in a browser) and look around before assuming there's only one image field.

`src/pages/NotFoundPage.jsx` (new)

```
import { Link } from "react-router-dom";

function NotFoundPage() {
  return (
    <div className="status">
      <p>There's nothing here.</p>
      <Link to="/" className="back-link">← Back to list</Link>
    </div>
  );
}

export default NotFoundPage;
```

`src/components/PokemonList.jsx` (update — items become links)

Only the `return` at the bottom changes; the fetch logic is untouched.

```
import { useState, useEffect } from "react";
import { Link } from "react-router-dom";

function getIdFromUrl(url) {
  const parts = url.split("/").filter(Boolean);
  return parts[parts.length - 1];
}

function capitalize(name) {
  return name.charAt(0).toUpperCase() + name.slice(1);
}

function getSpriteUrl(id) {
  return
  `https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/
  ${id}.png`;
}

function PokemonList() {
  const [pokemons, setPokemons] = useState([]);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    async function loadPokemons() {
      setIsLoading(true);

```

```

        setError(null);

        try {
            const response = await fetch("https://pokeapi.co/api/v2/pokemon?
limit=20");

            if (!response.ok) {
                throw new Error(`Server responded with status
${response.status}`);
            }

            const data = await response.json();
            setPokemons(data.results);
        } catch (err) {
            setError(err.message);
        } finally {
            setIsLoading(false);
        }
    }

    loadPokemons();
}, []);

if (isLoading) {
    return <p className="status">Loading Pokémon...</p>;
}

if (error) {
    return <p className="status status-error">Couldn't load the list:
{error}</p>;
}

return (
    <ul className="pokemon-list">
        {pokemons.map((pokemon) => {
            const id = getIdFromUrl(pokemon.url);
            return (
                <li key={pokemon.name} className="pokemon-list-item">
                    <Link to={` /pokemon/${pokemon.name}`} className="pokemon-
link">
                        <img
                            className="pokemon-sprite"
                            src={getSpriteUrl(id)}
                            alt={pokemon.name}
                            width={48}
                            height={48}
                        />
                        <span className="pokemon-id">#{id.padStart(3, "0")}</span>
                        <span className="pokemon-name">{capitalize(pokemon.name)}
</span>
                    </Link>
                </li>
            );
        })}
    </ul>
);

```

```
    </ul>
  );
}

export default PokemonList;
```

src/components/SearchForm.jsx (simplify — it navigates now, it doesn't fetch)

```
import { useState } from "react";
import { useNavigate } from "react-router-dom";

function SearchForm() {
  const [query, setQuery] = useState("");
  const [error, setError] = useState(null);
  const navigate = useNavigate();

  function handleSubmit(event) {
    event.preventDefault();

    const name = query.trim().toLowerCase();

    if (name === "") {
      setError("Type a Pokémon name first.");
      return;
    }

    setError(null);
    navigate(`/pokemon/${name}`);
  }

  return (
    <div className="search">
      <form onSubmit={handleSubmit} className="search-form">
        <input
          type="text"
          value={query}
          onChange={(event) => setQuery(event.target.value)}
          placeholder="Search a Pokémon by name..."
          className="search-input"
        />
        <button type="submit" className="search-button">
          Search
        </button>
      </form>

      {error && <p className="status status-error">{error}</p>}
    </div>
  );
}

export default SearchForm;
```

Point this out explicitly in class: compare this to S22's version. All the `isLoading`, `result`, and the fetch itself are *gone* — not simplified, deleted. That logic didn't disappear, it **moved** to `DetailPage`, because showing one Pokémon's data is now that page's job, not the search box's. `SearchForm`'s only remaining responsibility is figuring out *where to send the user*; it doesn't need to know whether that Pokémon actually exists — if it doesn't, `DetailPage`'s own fetch will 404 and show its own error message, which you already built. This is worth naming directly: **each piece now does exactly one job**, and the "does this Pokémon exist" check naturally lives in exactly one place instead of two.

`src/index.css` (updates — replace `.pokemon-list-item`'s rule, add the rest)

```
.pokemon-list-item {
  /* visual styling moved to .pokemon-link below, since that's the
  clickable card now */
}

.pokemon-link {
  display: flex;
  align-items: center;
  gap: 12px;
  background: white;
  border-radius: 8px;
  padding: 12px 16px;
  box-shadow: 0 1px 2px rgba(0, 0, 0, 0.08);
  text-decoration: none;
  color: inherit;
}

.pokemon-link:hover {
  box-shadow: 0 2px 6px rgba(0, 0, 0, 0.12);
}

.back-link {
  display: inline-block;
  margin-bottom: 16px;
  color: #cc0000;
  text-decoration: none;
  font-weight: 600;
}

.back-link:hover {
  text-decoration: underline;
}

.detail-page {
  text-align: center;
}

.detail-page img {
  margin: 0 auto;
}
```



```
.stat-list {
  list-style: none;
  padding: 0;
  margin: 16px auto 0;
  max-width: 240px;
  text-align: left;
}

.stat-list li {
  display: flex;
  justify-content: space-between;
  padding: 6px 0;
  border-bottom: 1px solid #eee;
}

.stat-name {
  text-transform: capitalize;
  color: #666;
}

.stat-value {
  font-weight: 600;
}
```

Run it

```
npm run dev
```

Try all of these:

- Click a Pokémon in the list → URL becomes `.../#/pokemon/<name>`, its detail page shows sprite, types, and stats.
- Press the browser's **back** button → returns to the list, exactly as expected.
- Search for a name that doesn't exist → lands on its detail page URL, which shows the "No Pokémon named..." error — not a crash.
- Manually edit the URL to something nonsensical, like `.../#/nonsense` → the 404 page appears.
- **The race condition, on purpose:** click into a Pokémon, then *immediately* click "← Back to list" and click a different Pokémon before the first one has visibly finished loading. Confirm the page settles on the *second* Pokémon you picked, not the first. If you comment out the `isCurrent` guard and try the same fast double-click a few times, you should occasionally catch it briefly flashing the wrong Pokémon before settling — put it back afterward.

Try this yourself: add a second `useEffect(() => { document.title = ...}, [pokemon])` inside `DetailPage` that sets the browser tab's title to the Pokémon's name once it loads. Small, but it's good practice recognizing "this is also a side effect, outside React" the moment you see it.

Step 4: the finished skeleton

What's new today

pokedex-mini/	
├ vite.config.js	← updated: base path added
├ package.json	← updated: deploy scripts added
└ src/	
├ App.jsx	← updated: nested routes under Layout
├ config.js	← new: API_BASE_URL, SPRITE_BASE_URL
└ utils.js	← new: capitalize, getIdFromUrl,
getSpriteUrl	
├ index.css	← updated: header styles
├ components/	
├ Layout.jsx	← new: shared header + <Outlet />
├ PokemonList.jsx	← updated: uses config.js + utils.js
└ SearchForm.jsx	← unchanged since S23
└ pages/	
├ ListPage.jsx	← unchanged since S23
├ DetailPage.jsx	← updated: uses config.js + utils.js
└ NotFoundPage.jsx	← unchanged since S23

src/config.js (new)

```
export const API_BASE_URL = "https://pokeapi.co/api/v2";
export const SPRITE_BASE_URL =

"https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon"
;
```

src/utils.js (new)

```
import { SPRITE_BASE_URL } from "./config.js";

export function getIdFromUrl(url) {
  // url looks like "https://pokeapi.co/api/v2/pokemon/25/"
  const parts = url.split("/").filter(Boolean);
  return parts[parts.length - 1];
}

export function capitalize(name) {
  return name.charAt(0).toUpperCase() + name.slice(1);
}

export function getSpriteUrl(id) {
  return `${SPRITE_BASE_URL}/${id}.png`;
}
```

src/components/Layout.jsx (new)

```
import { Outlet, Link } from "react-router-dom";

function Layout() {
  return (
    <div className="app">
      <header className="app-header">
        <Link to="/" className="app-title-link">
          <h1>PokéDex Mini</h1>
        </Link>
      </header>
      <main>
        <Outlet />
      </main>
    </div>
  );
}

export default Layout;
```

src/App.jsx (update — nested routes under Layout)

```
import { HashRouter, Routes, Route } from "react-router-dom";
import Layout from "../components/Layout.jsx";
import ListPage from "../pages/ListPage.jsx";
import DetailPage from "../pages/DetailPage.jsx";
import NotFoundPage from "../pages/NotFoundPage.jsx";

function App() {
  return (
    <HashRouter>
      <Routes>
        <Route element={<Layout />}>
          <Route path="/" element={<ListPage />} />
          <Route path="/pokemon/:name" element={<DetailPage />} />
          <Route path="*" element={<NotFoundPage />} />
        </Route>
      </Routes>
    </HashRouter>
  );
}

export default App;
```

src/components/PokemonList.jsx (update — imports instead of local helpers)

```
import { useState, useEffect } from "react";
import { Link } from "react-router-dom";
import { API_BASE_URL } from "../config.js";
import { getIdFromUrl, capitalize, getSpriteUrl } from "../utils.js";

function PokemonList() {
  const [pokemons, setPokemons] = useState([]);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    async function loadPokemons() {
      setIsLoading(true);
      setError(null);

      try {
        const response = await fetch(`${API_BASE_URL}/pokemon?limit=20`);

        if (!response.ok) {
          throw new Error(`Server responded with status ${response.status}`);
        }

        const data = await response.json();
        setPokemons(data.results);
      } catch (err) {
        setError(err.message);
      } finally {
        setIsLoading(false);
      }
    }

    loadPokemons();
  }, []);

  if (isLoading) {
    return <p className="status">Loading Pokémon...</p>;
  }

  if (error) {
    return <p className="status status-error">Couldn't load the list: {error}</p>;
  }

  return (
    <ul className="pokemon-list">
      {pokemons.map((pokemon) => {
        const id = getIdFromUrl(pokemon.url);
        return (
          <li key={pokemon.name} className="pokemon-list-item">
            <Link to={`/pokemon/${pokemon.name}`} className="pokemon-link">
              <img
```

```

        className="pokemon-sprite"
        src={getSpriteUrl(id)}
        alt={pokemon.name}
        width={48}
        height={48}
      />
      <span className="pokemon-id">#{id.padStart(3, "0")}</span>
      <span className="pokemon-name">{capitalize(pokemon.name)}
    </span>
  </Link>
</li>
);
}}}
</ul>
);
}

export default PokemonList;

```

src/pages/DetailPage.jsx (update — imports instead of local helpers)

```

import { useState, useEffect } from "react";
import { useParams, Link } from "react-router-dom";
import { API_BASE_URL } from "../config.js";
import { capitalize } from "../utils.js";

function DetailPage() {
  const { name } = useParams();
  const [pokemon, setPokemon] = useState(null);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    let isCurrent = true;

    async function loadPokemon() {
      setIsLoading(true);
      setError(null);
      setPokemon(null);

      try {
        const response = await fetch(`${API_BASE_URL}/pokemon/${name}`);

        if (!response.ok) {
          throw new Error(`No Pokémon named "${name}" — check the spelling.`);
        }

        const data = await response.json();

        if (isCurrent) {

```

```

        setPokemon(data);
      }
    } catch (err) {
      if (isCurrent) {
        setError(err.message);
      }
    } finally {
      if (isCurrent) {
        setIsLoading(false);
      }
    }
  }
}

loadPokemon();

return () => {
  isCurrent = false;
};
}, [name]);

if (isLoading) return <p className="status">Loading {name}...</p>;
if (error) return <p className="status status-error">{error}</p>;

return (
  <div className="detail-page">
    <Link to="/" className="back-link">← Back to list</Link>
    <img
      src={pokemon.sprites.other["official-artwork"].front_default}
      alt={pokemon.name}
      width={200}
      height={200}
    />
    <h2>{capitalize(pokemon.name)}</h2>
    <p className="pokemon-types">
      {pokemon.types.map((t) => t.type.name).join(", ")}
    </p>
    <ul className="stat-list">
      {pokemon.stats.map((s) => (
        <li key={s.stat.name}>
          <span className="stat-name">{s.stat.name}</span>
          <span className="stat-value">{s.base_stat}</span>
        </li>
      ))}
    </ul>
  </div>
);
}

export default DetailPage;

```

`SearchForm.jsx`, `ListPage.jsx`, and `NotFoundPage.jsx` don't change at all today — they didn't hardcode any API URL and didn't duplicate any helper, so there's nothing for today's cleanup to touch.

vite.config.js (update)

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  base: '/pokedex-mini/', // replace with your own repo name
  plugins: [react()],
})
```

src/index.css (append)

```
.app-header {
  margin-bottom: 8px;
}

.app-title-link {
  text-decoration: none;
  color: inherit;
}
```

Run it, then ship it

This is the hands-on sequence — do these in order, in your terminal, inside `pokedex-mini`.

1. Confirm the app still runs locally

```
npm run dev
```

Everything should behave exactly as it did at the end of S23 — today's changes so far are structural (config, layout, imports), nothing about what the app *does* should look different yet.

2. Add the deploy scripts to `package.json`

Open `package.json` and add `predeploy` and `deploy` alongside the scripts Vite already generated:

```
"scripts": {
  "dev": "vite",
  "build": "vite build",
  "lint": "eslint .",
  "preview": "vite preview",
  "predeploy": "npm run build",
  "deploy": "gh-pages -d dist"
}
```

3. Put the project on GitHub

`gh-pages` (next step) needs somewhere to push to, so the project needs a GitHub repository connected before you can deploy. If you haven't already been committing this project since S21, create an empty repository on GitHub first (**no** README, **no** `.gitignore` — Vite already generated one for you), then, from inside `pokedex-mini`:

```
# 1. make sure you're in the project folder
cd pokedex-mini

# 2. initialize Git in this project
git init

# 3. connect this project to its own GitHub repository
git remote add origin https://github.com/<your-username>/pokedex-mini.git

# 4. commit and push everything you've built across S21-S24
git add .
git commit -m "Initial commit"
git branch -M main
git push -u origin main
```

If this project already has a Git history from earlier sessions, skip straight to committing today's changes and `git push` — there's no need to `git init` a second time.

4. Install `gh-pages` and deploy

```
npm install --save-dev gh-pages
npm run deploy
```

`predeploy` runs first automatically (building the app into `dist/`), then `deploy` pushes that built output to a `gh-pages` branch on the `origin` remote you just connected in step 3.

5. Turn on GitHub Pages

On GitHub, go to **Settings** → **Pages**, and set the source to the `gh-pages` branch. Wait a minute or two, then visit <https://<your-username>.github.io/pokedex-mini/> and walk through Section 6's refresh test on the live URL.

6. Re-deploying after future changes

`npm run deploy` only pushes the *built* app — it never touches your source code's history on `main`. From now on, a normal round of changes needs both:

```
git add .
git commit -m "... "
git push          # updates your source code on GitHub (what your
```



```
instructor reads)  
npm run deploy    # updates the live site (what visitors see)
```

Forgetting `npm run deploy` is the most common reason a project's source looks up to date on GitHub while the live demo still shows an older version.

Try this yourself: temporarily change `base` in `vite.config.js` to the wrong string (e.g. `/wrong-name/`), commit, and run `npm run deploy` again. Look at what breaks on the live site and what the browser console says. Then fix it back and redeploy. Recognizing this specific failure on purpose, once, makes it far less confusing the first time it happens by accident.
